---

1. Need of PL/pgSQL Block (When SQL Already Exists)

1.1 Introduction

SQL (DDL, DML, DQL) is **declarative** — it tells *what to do*.
PL/pgSQL is **procedural** — it tells *how to do it step by step*.

👉 PL/pgSQL is needed when:

- You need **conditions (IF/ELSE)**
- You need **loops**
- You need **variables**
- You need **complex business logic**

1.2 Basic Examples (Without DML/DDL)

Example 1

```
DO $$
DECLARE
  v_num INT := 10;
BEGIN
  IF v_num > 5 THEN
    RAISE NOTICE 'Greater than 5';
  END IF;
END $$;
```

Example 2

```
DO $$
DECLARE
  v_text TEXT := 'PostgreSQL';
BEGIN
  RAISE NOTICE 'Welcome to %', v_text;
END $$;
```

1.3 Advanced Examples (With SQL + Aggregates + DML)

Example 1 (Using COUNT)

```
DO $$
DECLARE
  total_users INT;
BEGIN
  SELECT COUNT(*) INTO total_users FROM users;
```

```
  IF total_users > 0 THEN
    RAISE NOTICE 'Users exist';
  END IF;
END $$;
```

Example 2 (INSERT with condition)

```
DO $$
DECLARE
  v_count INT;
BEGIN
  SELECT COUNT(*) INTO v_count FROM employees;

  IF v_count = 0 THEN
    INSERT INTO employees(name) VALUES ('Admin');
  END IF;
END $$;
```

Example 3 (UPDATE with condition)

```
DO $$
BEGIN
  UPDATE employees
  SET salary = salary + 1000
  WHERE salary < 5000;
END $$;
```

---

2. Structure of PL/pgSQL Block (Must vs Optional)

2.1 Basic Examples

Example 1 (Minimal Block)

```
DO $$
BEGIN
  NULL;
END $$;
```

Example 2 (With Declare)

```
DO $$
DECLARE
  v_num INT := 1;
BEGIN
  RAISE NOTICE '%', v_num;
```

```
END $$;
```

2.2 Advanced Examples

Example 1 (Full Block with Exception)

```
DO $$
DECLARE
  v_result INT;
BEGIN
  v_result := 10 / 0;

EXCEPTION
  WHEN division_by_zero THEN
    RAISE NOTICE 'Error occurred';
END $$;
```

Example 2 (With SELECT INTO)

```
DO $$
DECLARE
  avg_salary NUMERIC;
BEGIN
  SELECT AVG(salary) INTO avg_salary FROM employees;
  RAISE NOTICE 'Average: %', avg_salary;
END $$;
```

Example 3 (Nested Block)

```
DO $$
BEGIN
  DECLARE v_inner INT := 5;
  BEGIN
    RAISE NOTICE '%', v_inner;
  END;
END $$;
```

---

3. Block Processing (DO, DECLARE, BEGIN...END)

3.1 Basic Examples

Example 1

```
DO $$
BEGIN
```

```
  RAISE NOTICE 'Hello World';
END $$;
```

Example 2

```
DO $$
DECLARE
  v_counter INT := 1;
BEGIN
  RAISE NOTICE 'Counter: %', v_counter;
END $$;
```

3.2 Advanced Examples

Example 1 (Using SUM)

```
DO $$
DECLARE
  total_salary NUMERIC;
BEGIN
  SELECT SUM(salary) INTO total_salary FROM employees;
  RAISE NOTICE '%', total_salary;
END $$;
```

Example 2 (DELETE with condition)

```
DO $$
BEGIN
  DELETE FROM employees WHERE salary < 3000;
END $$;
```

Example 3 (Insert + Aggregate)

```
DO $$
DECLARE
  max_salary NUMERIC;
BEGIN
  SELECT MAX(salary) INTO max_salary FROM employees;

  INSERT INTO audit_log(message)
  VALUES ('Max salary is ' || max_salary);
END $$;
```

---

4. Conditional Logic (IF, ELSE, ELSIF, CASE)

4.1 Basic Examples

Example 1 (IF)

```
DO $$
DECLARE v_num INT := 20;
BEGIN
  IF v_num > 10 THEN
    RAISE NOTICE 'Big';
  END IF;
END $$;
```

Example 2 (CASE)

```
DO $$
DECLARE v_score INT := 70;
BEGIN
  RAISE NOTICE '%',
  CASE
    WHEN v_score >= 50 THEN 'Pass'
    ELSE 'Fail'
  END;
END $$;
```

4.2 Advanced Examples

Example 1 (CASE in SELECT)

```
SELECT name,
CASE
  WHEN salary > 5000 THEN 'High'
  ELSE 'Low'
END
FROM employees;
```

Example 2 (IF with COUNT)

```
DO $$
DECLARE total INT;
BEGIN
  SELECT COUNT(*) INTO total FROM employees;

  IF total > 10 THEN
    RAISE NOTICE 'Large company';
  END IF;
```

```
END $$;
```

Example 3 (UPDATE with CASE)

```
UPDATE employees
SET bonus =
CASE
  WHEN salary > 5000 THEN 1000
  ELSE 500
END;
```

---

5. Loops (LOOP, WHILE, FOR)

5.1 Basic Examples

Example 1 (LOOP)

```
DO $$
DECLARE i INT := 1;
BEGIN
  LOOP
    EXIT WHEN i > 3;
    RAISE NOTICE '%', i;
    i := i + 1;
  END LOOP;
END $$;
```

Example 2 (FOR)

```
DO $$
BEGIN
  FOR i IN 1..3 LOOP
    RAISE NOTICE '%', i;
  END LOOP;
END $$;
```

5.2 Advanced Examples

Example 1 (FOR with SELECT)

```
DO $$
DECLARE rec RECORD;
BEGIN
  FOR rec IN SELECT name FROM employees LOOP
    RAISE NOTICE '%', rec.name;
```

```
  END LOOP;
END $$;
```

Example 2 (WHILE + UPDATE)

```
DO $$
DECLARE v_id INT := 1;
BEGIN
  WHILE v_id <= 5 LOOP
    UPDATE employees SET salary = salary + 100 WHERE id = v_id;
    v_id := v_id + 1;
  END LOOP;
END $$;
```

Example 3 (Aggregate inside loop)

```
DO $$
DECLARE total NUMERIC := 0;
rec RECORD;
BEGIN
  FOR rec IN SELECT salary FROM employees LOOP
    total := total + rec.salary;
  END LOOP;

  RAISE NOTICE 'Total: %', total;
END $$;
```

---

6. Core and Advanced SQL in PostgreSQL

6.1 Basic Examples

Example 1

```
SELECT 10 + 5;
```

Example 2

```
SELECT CURRENT_DATE;
```

6.2 Advanced Examples

Example 1 (Aggregate Functions)

```
SELECT AVG(salary), MAX(salary), MIN(salary)
FROM employees;
```

Example 2 (GROUP BY)

```
SELECT department_id, COUNT(*)
```

FROM employees

GROUP BY department_id;

Example 3 (JOIN)

SELECT e.name, d.department_name

FROM employees e

JOIN departments d

ON e.department_id = d.id;

Example 4 (Subquery)

SELECT name

FROM employees

WHERE salary > (SELECT AVG(salary) FROM employees);

---

7. Conclusion

- SQL handles **data operations**
- PL/pgSQL handles **logic and control flow**
- Both together create **powerful database applications**

---

8. Final Summary

- Use SQL → for querying and data manipulation
- Use PL/pgSQL → for logic, automation, and control
- Use both → for real-world applications

PL/pgSQL Report: Difference Between IF and WHEN in PostgreSQL

1. Introduction

PostgreSQL provides two main ways to implement conditional logic:

- IF statement (procedural logic)
- WHEN clause (used inside SQL/PLpgSQL structures)

Although they appear similar, they serve **different purposes** and are **not interchangeable**.

---

2. Overview of IF Statement

Definition

IF is a **control flow statement** used in PL/pgSQL to execute code conditionally.

Syntax

```
IF condition THEN
  statements;
ELSIF condition THEN
  statements;
ELSE
  statements;
END IF;
```

Example

```
DO $$
DECLARE
  v_num INT := 10;
BEGIN
  IF v_num > 5 THEN
    RAISE NOTICE 'Greater than 5';
  ELSE
    RAISE NOTICE 'Less or equal to 5';
  END IF;
END $$;
```

Key Characteristics

- Used in **procedural blocks**
- Can execute **multiple statements**
- Supports complex logic
- Cannot be used in SQL queries

---

3. Overview of WHEN Clause

Definition

WHEN is a **conditional clause** used inside predefined SQL or PL/pgSQL structures.

Where WHEN is Used

1. CASE statements
2. EXCEPTION handling
3. TRIGGER conditions

---

4. WHEN in Different Structures

4.1 CASE Structure

```
SELECT
  CASE
    WHEN age >= 18 THEN 'Adult'
    ELSE 'Minor'
  END
FROM users;
```

👉 Returns a value based on conditions.

4.2 EXCEPTION Handling

```
DO $$
BEGIN
  1 / 0;
EXCEPTION
  WHEN division_by_zero THEN
    RAISE NOTICE 'Cannot divide by zero';
END $$;
```

👉 Handles specific runtime errors.

4.3 Trigger Condition

```
CREATE TRIGGER check_age
BEFORE INSERT ON users
FOR EACH ROW
WHEN (NEW.age > 18)
EXECUTE FUNCTION validate_user();
```

👉 Filters when a trigger should execute.

---

5. Key Differences Between IF and WHEN

Feature	IF	WHEN
Type	Statement	Clause
Usage	Procedural logic	Inside structures
Standalone	Yes	No
Executes multiple statements	Yes	No
Used in SQL queries	No	Yes
Returns value	No	Yes (via CASE)

---

6. Can IF Be Used Instead of WHEN?

❌ **No (in SQL queries)**

```
SELECT IF salary > 5000 THEN 'High';  -- INVALID
```

✅ **Correct Approach**

```
SELECT
  CASE
    WHEN salary > 5000 THEN 'High'
  END
FROM employees;
```

---

7. Limitations of IF

- Cannot be used in:
  - SELECT statements
  - WHERE, GROUP BY, ORDER BY
- Only works inside:
  - Functions
  - Procedures
  - DO blocks
- Cannot directly return values in SQL queries

---

8. Limitations of WHEN

- Cannot be used independently
- Cannot execute multiple statements
- Only works inside structures like:
  - CASE
  - EXCEPTION
  - TRIGGER
- Designed for **value-based logic**, not procedural control

---

9. Practical Comparison

Using CASE (WHEN)

```
SELECT
  CASE
    WHEN salary > 5000 THEN 'High'
    ELSE 'Low'
  END
FROM employees;
```

Using IF

```
DO $$
DECLARE
  v_salary INT := 6000;
BEGIN
  IF v_salary > 5000 THEN
    RAISE NOTICE 'High';
  ELSE
    RAISE NOTICE 'Low';
  END IF;
END $$;
```

---

10. Conceptual Difference

- IF → Controls **execution flow**
- WHEN → Defines **conditions within expressions/structures**

---

11. Internal Working (Important Concept)

PostgreSQL has two engines:

SQL Engine

- Handles queries
- Uses CASE and WHEN

PL/pgSQL Engine

- Handles procedural code
- Uses IF, loops, variables

---

12. Conclusion

- IF is used for **procedural decision-making**
- WHEN is used for **conditional expressions within structures**

- They serve different purposes and cannot replace each other

---

13. One-Line Summary

IF is a procedural control statement used in PL/pgSQL blocks, whereas WHEN is a conditional clause used inside SQL structures like CASE, EXCEPTION, and triggers.

Advanced PL/pgSQL Report Using DVD Rental Database

---

1. Introduction

The **DVD Rental database** is a widely used PostgreSQL sample database that simulates a real-world rental system. It contains tables such as:

- customer
- film
- rental
- payment
- inventory
- category

This report demonstrates **advanced PL/pgSQL block usage** using:

- DML (INSERT, UPDATE, DELETE)
- DDL (CREATE, ALTER)
- Conditional logic (IF, CASE)
- Loops (FOR, WHILE, LOOP)
- Aggregates (COUNT, SUM, AVG)

---

2. Scenario 1: Conditional Insert Based on Customer Activity

Objective

Insert customers into a loyal_customers table if they have more than 30 rentals.

DDL Setup

```
CREATE TABLE loyal_customers (
  customer_id INT,
  total_rentals INT
);
```

PL/pgSQL Block

```
DO $$
DECLARE
  rec RECORD;
  rental_count INT;
BEGIN
  FOR rec IN SELECT customer_id FROM customer LOOP

    SELECT COUNT(*) INTO rental_count
    FROM rental
```

```
    WHERE customer_id = rec.customer_id;

    IF rental_count > 30 THEN
      INSERT INTO loyal_customers
      VALUES (rec.customer_id, rental_count);
    END IF;

  END LOOP;
END $$;
```

---

3. Scenario 2: Conditional Update Using CASE

Objective

Update film rental rates based on film length.

```
DO $$
BEGIN
  UPDATE film
  SET rental_rate =
  CASE
    WHEN length > 120 THEN rental_rate + 2
    WHEN length > 90 THEN rental_rate + 1
    ELSE rental_rate
  END;
END $$;
```

---

4. Scenario 3: Loop Through Rentals and Calculate Revenue

Objective

Calculate total revenue per customer using loop.

```
DO $$
DECLARE
  rec RECORD;
  total_payment NUMERIC;
BEGIN
  FOR rec IN SELECT customer_id FROM customer LOOP

    SELECT SUM(amount) INTO total_payment
```

```
    FROM payment
    WHERE customer_id = rec.customer_id;

    RAISE NOTICE 'Customer % → Total Payment: %',
    rec.customer_id, total_payment;

  END LOOP;
END $$;
```

---

5. Scenario 4: Conditional Deletion Based on Inactivity

Objective

Delete customers who have no rentals.

```
DO $$
DECLARE
  rec RECORD;
  rental_count INT;
BEGIN
  FOR rec IN SELECT customer_id FROM customer LOOP

    SELECT COUNT(*) INTO rental_count
    FROM rental
    WHERE customer_id = rec.customer_id;

    IF rental_count = 0 THEN
      DELETE FROM customer WHERE customer_id = rec.customer_id;
    END IF;

  END LOOP;
END $$;
```

---

6. Scenario 5: Dynamic Table Creation (DDL in PL/pgSQL)

Objective

Create a summary table dynamically.

```
DO $$
BEGIN
```

```
  EXECUTE '
  CREATE TABLE IF NOT EXISTS customer_summary (
    customer_id INT,
    total_rentals INT,
    total_payment NUMERIC
  )';
END $$;
```

---

7. Scenario 6: Populate Summary Table Using Loop + Aggregates

---

```
DO $$
DECLARE
  rec RECORD;
  total_rentals INT;
  total_payment NUMERIC;
BEGIN
  FOR rec IN SELECT customer_id FROM customer LOOP

    SELECT COUNT(*) INTO total_rentals
    FROM rental
    WHERE customer_id = rec.customer_id;

    SELECT SUM(amount) INTO total_payment
    FROM payment
    WHERE customer_id = rec.customer_id;

    INSERT INTO customer_summary
    VALUES (rec.customer_id, total_rentals, total_payment);

  END LOOP;
END $$;
```

---

8. Scenario 7: WHILE Loop for Batch Processing

Objective

Increase rental rate for first 10 films.

```
DO $$
```

```
DECLARE
  counter INT := 1;
BEGIN
  WHILE counter <= 10 LOOP

    UPDATE film
    SET rental_rate = rental_rate + 0.5
    WHERE film_id = counter;

    counter := counter + 1;
  END LOOP;
END $$;
```

---

9. Scenario 8: Exception Handling with Logging

Objective

Handle errors during updates.

```
DO $$
BEGIN
  UPDATE film SET rental_rate = rental_rate / 0;

EXCEPTION
  WHEN division_by_zero THEN
    RAISE NOTICE 'Error: Division by zero occurred';
END $$;
```

---

10. Scenario 9: Conditional Logic with Multiple Criteria

Objective

Categorize customers based on spending.

```
DO $$
DECLARE
  rec RECORD;
  total NUMERIC;
  category TEXT;
BEGIN
  FOR rec IN SELECT customer_id FROM customer LOOP
```

```
    SELECT SUM(amount) INTO total
    FROM payment
    WHERE customer_id = rec.customer_id;

    category :=
    CASE
      WHEN total > 200 THEN 'Premium'
      WHEN total > 100 THEN 'Gold'
      ELSE 'Regular'
    END;

    RAISE NOTICE 'Customer % → %',
    rec.customer_id, category;

  END LOOP;
END $$;
```

---

11. Scenario 10: Nested Blocks and Advanced Logic

```
DO $$
DECLARE
  rec RECORD;
BEGIN
  FOR rec IN SELECT film_id, rental_rate FROM film LOOP

    BEGIN
      IF rec.rental_rate < 1 THEN
        UPDATE film
        SET rental_rate = rental_rate + 1
        WHERE film_id = rec.film_id;
      END IF;

    EXCEPTION
      WHEN OTHERS THEN
        RAISE NOTICE 'Error updating film %', rec.film_id;
    END;
```

```
    END LOOP;
END $$;
```

---

12. Key Concepts Demonstrated

Concept	Used In
FOR LOOP	Iterating customers/films
WHILE LOOP	Batch updates
IF / CASE	Conditional logic
DML	INSERT, UPDATE, DELETE
DDL	CREATE TABLE
Aggregates	COUNT, SUM
Exception Handling	Error management
Dynamic SQL	EXECUTE

---

13. Conclusion

This report demonstrates that PL/pgSQL:

- Enables **automation of complex workflows**
- Combines **SQL power with programming logic**
- Supports **real-world business scenarios**

Using the DVD Rental database, we implemented:

- Conditional insertions
- Loop-based processing
- Dynamic table creation
- Advanced decision-making logic

---

14. Final Insight

PL/pgSQL transforms PostgreSQL from a query engine into a full-fledged programming environment capable of handling enterprise-level logic directly inside the database.
